

NAME:VIMAL S 23BCE0345

NAME:KAARTHIKEYAN S S 23BCE2041

->addExpenseFunction

```
import json
import boto3
import uuid
from decimal import Decimal
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Expenses')
sns = boto3.client('sns')
SNS_TOPIC_ARN = "arn:aws:sns:us-east-1:097556381774:ExpenseAlertTopic"
def lambda_handler(event, context):
    try:
        if "body" in event:
            body = json.loads(event["body"])
        else:
            body = event
        expense_id = str(uuid.uuid4())
        amount = Decimal(str(body["amount"]))
        category = body["category"]
        date = body["date"]
        table.put_item(
            Item={
                "expense_id": expense_id,
                "amount": amount,
                "category": category,
                "date": date
            }
        )
        if amount > 5000:
            sns.publish(
                TopicArn=SNS_TOPIC_ARN,
                Subject=" Expense Alert: Over ₹5000",
                Message=f" High Expense Alert!\nAmount: ₹{amount}\nCategory: {category}\nDate: {date}"
            )
        return {
            "statusCode": 200,
            "body": json.dumps(" Expense added successfully!")
        }
```

```

except Exception as e:
    return {
        "statusCode": 500,
        "body": json.dumps(f" Error: {str(e)}")
    }

```

->deleteExpenseFunction

```

import json
import boto3
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Expenses')
def lambda_handler(event, context):
    try:
        if "body" in event:
            body = json.loads(event["body"])
        else:
            body = event
        expense_id = body["expense_id"]
        table.delete_item(
            Key={
                "expense_id": expense_id
            }
        )
        return {
            "statusCode": 200,
            "body": json.dumps("Expense deleted successfully!")
        }
    except Exception as e:
        return {
            "statusCode": 500,
            "body": json.dumps(f"Error: {str(e)}")
        }

```

->getExpenseFunction

```

import json
import boto3
from decimal import Decimal
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Expenses')
class DecimalEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, Decimal):

```

```

return float(obj)
return super(DecimalEncoder, self).default(obj)
def lambda_handler(event, context):
try:
response = table.scan()
items = response.get("Items", [])
return {
37"statusCode": 200,
"headers": {
"Access-Control-Allow-Origin": "*",
"Content-Type": "application/json"
},
"body": json.dumps(items, cls=DecimalEncoder)
}
except Exception as e:
return {
"statusCode": 500,
"body": json.dumps(f"Error: {str(e)}")
}

```

->budgetFunction

```

import json
import boto3
import os
from decimal import Decimal
from boto3.dynamodb.conditions import Key, Attr
from datetime import datetime
dynamodb = boto3.resource('dynamodb')
sns = boto3.client('sns')
table = dynamodb.Table(os.environ['DYNAMODB_TABLE'])
sns_topic = os.environ.get('SNS_TOPIC_ARN')
def lambda_handler(event, context):
try:
http_method = event['httpMethod']
body = json.loads(event['body']) if event.get('body') else {}
if http_method == 'POST':
return set_budget(body)
elif http_method == 'GET':
return check_budget(body)
else:
return response(400, {"error": "Unsupported HTTP method"})
except Exception as e:
38return response(500, {"error": str(e)})

```

```

def set_budget(data):
    """Sets or updates the monthly budget for a user."""
    user_id = data['user_id']
    month = data['month'] # Format: "YYYY-MM"
    limit = Decimal(str(data['limit']))
    item = {
        'UserID': user_id,
        'SortKey': f"BUDGET#{month}",
        'BudgetLimit': limit,
        'CreatedAt': datetime.utcnow().isoformat()
    }
    table.put_item(Item=item)
    return response(200, {"message": f"Budget set successfully for {month}", "limit": float(limit)})
def check_budget(data):
    """Checks total expenses against the monthly budget and triggers alert if exceeded."""
    user_id = data['user_id']
    month = data['month']
    budget_response = table.get_item(
        Key={'UserID': user_id, 'SortKey': f"BUDGET#{month}"})
    if 'Item' not in budget_response:
        return response(404, {"message": f"No budget found for {month}"})
    budget_limit = Decimal(str(budget_response['Item']['BudgetLimit']))
    start_key = f"EXPENSE#{month}"
    expenses = table.query(
        KeyConditionExpression=Key('UserID').eq(user_id) & Key('SortKey').begins_with(start_key)
    )
    total_spent = sum(Decimal(str(item['Amount'])) for item in expenses.get('Items', []))
    remaining = budget_limit - total_spent
    result = {
        "month": month,
        "budget_limit": float(budget_limit),
        "total_spent": float(total_spent),
        "remaining_budget": float(remaining),
        "status": "Within budget" if remaining >= 0 else "Exceeded"
    }
    if remaining < 0 and sns_topic:
        sns.publish(
            TopicArn=sns_topic,
            Subject="Budget Limit Exceeded!",
            Message=f"User {user_id} has exceeded the monthly budget for {month}. "
            f"Spent ₹{total_spent}, exceeding limit ₹{budget_limit}."
        )

```

```

return response(200, result)
def response(status_code, body):
    """Helper function for consistent HTTP responses."""
    return {
        "statusCode": status_code,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps(body)
    }

```

->Frontend

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Personal Expense Tracker</title>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/tesseract.js@4/dist/tesseract.min.js"></script>
</style>
:root {
  --primary-color: #272c33;
  --secondary-color: #415341;
  --danger-color: #4f3836;
  --warning-color: #433724;
  --light-bg: #f5f7fa;
  --white: #ffffff;
  --text-dark: #2c3e50;
  --text-light: #7f8c8d;
  --border-color: #e1e5e9;
}

* {
  box-sizing: border-box;
}

body {
  font-family: "Poppins", "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
  background-color: var(--light-bg);
  margin: 0;
  padding: 0;
  color: var(--text-dark);
  display: flex;
  flex-direction: column;
  align-items: center;
  min-height: 100vh;
}

.container {
  width: 100%;
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
}

```

```
}
```

```
#authContainer {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 100vh;  
  width: 100%;  
}
```

```
.auth-form {  
  width: 100%;  
  max-width: 400px;  
  background: var(--white);  
  border-radius: 12px;  
  padding: 30px;  
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);  
  border: 1px solid var(--border-color);  
}
```

```
.auth-form h1 {  
  text-align: center;  
  margin-bottom: 10px;  
  color: var(--primary-color);  
}
```

```
.auth-form p {  
  text-align: center;  
  margin-bottom: 25px;  
  color: var(--text-light);  
}
```

```
.auth-form input {  
  width: 100%;  
  padding: 12px 15px;  
  margin: 10px 0;  
  border: 1px solid var(--border-color);  
  border-radius: 8px;  
  background: var(--white);  
  color: var(--text-dark);  
  font-size: 16px;  
  transition: all 0.3s ease;  
  outline: none;  
}
```

```
.auth-form input:focus {  
  border-color: var(--primary-color);  
  box-shadow: 0 0 0 2px rgba(74, 144, 226, 0.2);  
}
```

```
.auth-form button {  
  width: 100%;  
  padding: 12px 15px;  
  border-radius: 8px;  
  border: none;  
  font-size: 16px;
```

```
font-weight: 600;
cursor: pointer;
transition: all 0.3s ease;
letter-spacing: 0.5px;
background: var(--primary-color);
color: white;
margin-top: 10px;
}

.auth-form button:hover {
background: #3a80d2;
transform: translateY(-2px);
box-shadow: 0 4px 8px rgba(74, 144, 226, 0.3);
}

.auth-switch {
text-align: center;
margin-top: 20px;
color: var(--text-light);
}

.auth-switch a {
color: var(--primary-color);
text-decoration: none;
font-weight: 600;
cursor: pointer;
}

.auth-switch a:hover {
text-decoration: underline;
}

#appContainer {
display: none;
width: 100%;
}

h1 {
font-size: 2.5rem;
color: var(--text-dark);
margin-bottom: 20px;
text-align: center;
}

h2 {
margin-top: 50px;
font-size: 1.5rem;
color: var(--text-dark);
text-align: center;
}

h3 {
font-size: 1.3rem;
color: var(--text-dark);
margin-bottom: 15px;
text-align: center;
}
```

```
}
```

```
form {  
  width: 100%;  
  max-width: 600px;  
  background: var(--white);  
  border-radius: 12px;  
  padding: 25px 30px;  
  margin: 20px auto;  
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);  
  border: 1px solid var(--border-color);  
  transition: transform 0.3s ease, box-shadow 0.3s ease;  
}
```

```
form:hover {  
  transform: translateY(-3px);  
  box-shadow: 0 8px 16px rgba(0, 0, 0, 0.1);  
}
```

```
input, select {  
  width: calc(33.33% - 12px);  
  padding: 12px 15px;  
  margin: 8px 6px;  
  border: 1px solid var(--border-color);  
  border-radius: 8px;  
  background: var(--white);  
  color: var(--text-dark);  
  font-size: 16px;  
  transition: all 0.3s ease;  
  outline: none;  
}
```

```
input::placeholder {  
  color: #a0a4a8;  
}
```

```
input:focus, select:focus {  
  border-color: var(--primary-color);  
  box-shadow: 0 0 0 2px rgba(74, 144, 226, 0.2);  
}
```

```
button {  
  padding: 12px 15px;  
  border-radius: 8px;  
  border: none;  
  font-size: 16px;  
  font-weight: 600;  
  cursor: pointer;  
  transition: all 0.3s ease;  
  letter-spacing: 0.5px;  
}
```

```
form#expenseForm button {  
  width: calc(33.33% - 12px);  
  background: var(--secondary-color);  
  color: white;
```



```
    box-shadow: 0 2px 5px rgba(76, 175, 80, 0.2);  
}
```

```
form#expenseForm button:hover {  
    background: #45a049;  
    transform: translateY(-2px);  
    box-shadow: 0 4px 8px rgba(76, 175, 80, 0.3);  
}
```

```
.delete-btn {  
    background: var(--danger-color);  
    color: white;  
    box-shadow: 0 2px 5px rgba(244, 67, 54, 0.2);  
}
```

```
.delete-btn:hover {  
    background: #e53935;  
    transform: translateY(-2px);  
    box-shadow: 0 4px 8px rgba(244, 67, 54, 0.3);  
}
```

```
.delete-all-btn {  
    width: 100%;  
    background: var(--danger-color);  
    color: white;  
    box-shadow: 0 2px 5px rgba(244, 67, 54, 0.2);  
    margin-top: 10px;  
}
```

```
.delete-all-btn:hover {  
    background: #634c4b;  
    transform: translateY(-2px);  
    box-shadow: 0 4px 8px rgba(244, 67, 54, 0.3);  
}
```

```
.budget-btn {  
    background: var(--primary-color);  
    color: white;  
    box-shadow: 0 2px 5px rgba(33, 150, 243, 0.2);  
}
```

```
.budget-btn:hover {  
    background: #1e88e5;  
    transform: translateY(-2px);  
    box-shadow: 0 4px 8px rgba(33, 150, 243, 0.3);  
}
```

```
.scan-btn {  
    background: var(--warning-color);  
    color: white;  
    box-shadow: 0 2px 5px rgba(255, 152, 0, 0.2);  
    width: 100%;  
    margin-top: 10px;  
}
```

```
.scan-btn:hover {
```

```
background: #e68900;
transform: translateY(-2px);
box-shadow: 0 4px 8px rgba(255, 152, 0, 0.3);
}
```

```
#deleteForm input {
  width: 65%;
}
```

```
#deleteForm button {
  width: 30%;
}
```

```
#budgetDisplay {
  width: 100%;
  max-width: 600px;
  background: var(--white);
  border-radius: 12px;
  padding: 20px 30px;
  margin: 20px auto;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
  text-align: center;
  color: var(--text-dark);
  font-size: 1.2rem;
  font-weight: 600;
  border: 1px solid var(--border-color);
}
```

```
#budgetDisplay span {
  font-size: 1.5rem;
}
```

```
.positive {
  color: var(--secondary-color);
}
```

```
.negative {
  color: var(--danger-color);
}
```

```
.warning {
  color: var(--warning-color);
}
```

```
#expenseList {
  width: 100%;
  max-width: 600px;
  margin: 20px auto;
}
```

```
#expenseList h2 {
  text-align: center;
  margin-bottom: 20px;
}
```

```
.expense-item {
```

```
background: var(--white);
border-radius: 10px;
padding: 15px 20px;
margin-bottom: 12px;
display: flex;
justify-content: space-between;
align-items: center;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.06);
border: 1px solid var(--border-color);
transition: transform 0.2s ease;
}
```

```
.expense-item:hover {
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}
```

```
.expense-details {
  display: flex;
  flex-direction: column;
}
```

```
.expense-category {
  font-weight: 600;
  font-size: 1.1rem;
  color: var(--text-dark);
  margin-bottom: 5px;
}
```

```
.expense-date {
  font-size: 0.9rem;
  color: var(--text-light);
}
```

```
.expense-amount {
  font-weight: 700;
  font-size: 1.2rem;
  color: var(--danger-color);
}
```

```
.expense-id {
  font-size: 0.8rem;
  color: #95a5a6;
  margin-top: 5px;
}
```

```
.empty-list {
  text-align: center;
  color: var(--text-light);
  font-style: italic;
  padding: 30px;
  background: var(--white);
  border-radius: 10px;
  border: 1px solid var(--border-color);
}
```

```
#predictionDisplay {
  width: 100%;
  max-width: 600px;
  background: var(--white);
  border-radius: 12px;
  padding: 20px 30px;
  margin: 20px auto;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
  text-align: center;
  color: var(--text-dark);
  font-size: 1rem;
  font-weight: 600;
  border: 1px solid var(--border-color);
}
```

```
.prediction-positive {
  color: var(--secondary-color);
}
```

```
.prediction-warning {
  color: var(--warning-color);
}
```

```
.prediction-danger {
  color: var(--danger-color);
}
```

```
.user-info {
  display: flex;
  justify-content: space-between;
  align-items: center;
  width: 100%;
  max-width: 600px;
  margin: 0 auto 20px;
}
```

```
.user-greeting {
  font-size: 1.2rem;
  font-weight: 600;
}
```

```
.logout-btn {
  background: var(--text-light);
  color: white;
  padding: 8px 15px;
  font-size: 0.9rem;
}
```

```
.logout-btn:hover {
  background: #6c7c7d;
}
```

```
.chart-container {
  width: 100%;
  max-width: 600px;
  background: var(--white);
}
```

```
border-radius: 12px;
padding: 25px 30px;
margin: 20px auto;
box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
border: 1px solid var(--border-color);
}
```

```
.chart-wrapper {
position: relative;
height: 300px;
width: 100%;
}
```

```
.scanner-container {
width: 100%;
max-width: 600px;
background: var(--white);
border-radius: 12px;
padding: 25px 30px;
margin: 20px auto;
box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
border: 1px solid var(--border-color);
}
```

```
.camera-preview {
width: 100%;
max-width: 400px;
height: 300px;
margin: 0 auto 20px;
border: 2px dashed var(--border-color);
border-radius: 8px;
display: flex;
align-items: center;
justify-content: center;
overflow: hidden;
background: #f9f9f9;
}
```

```
.camera-preview video,
.camera-preview canvas {
width: 100%;
height: 100%;
object-fit: cover;
}
```

```
.camera-placeholder {
text-align: center;
color: var(--text-light);
}
```

```
.scanner-controls {
display: flex;
gap: 10px;
justify-content: center;
margin-bottom: 15px;
}
```

```
.scanner-result {
  margin-top: 20px;
  padding: 15px;
  background: #f8f9fa;
  border-radius: 8px;
  border-left: 4px solid var(--primary-color);
}
```

```
.scanner-loading {
  display: none;
  text-align: center;
  padding: 20px;
}
```

```
.scanner-loading .spinner {
  border: 4px solid #f3f3f3;
  border-top: 4px solid var(--primary-color);
  border-radius: 50%;
  width: 40px;
  height: 40px;
  animation: spin 2s linear infinite;
  margin: 0 auto 10px;
}
```

```
@keyframes spin {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}
```

```
.password-strength {
  height: 5px;
  margin-top: 5px;
  border-radius: 5px;
  transition: all 0.3s ease;
}
```

```
.strength-weak {
  background-color: var(--danger-color);
  width: 25%;
}
```

```
.strength-medium {
  background-color: var(--warning-color);
  width: 50%;
}
```

```
.strength-strong {
  background-color: var(--secondary-color);
  width: 100%;
}
```

```
@media (max-width: 700px) {
  input, select, button {
    width: 100%;
    margin: 8px 0;
  }
}
```

```

}
#deleteForm input,
#deleteForm button {
  width: 100%;
}

form#expenseForm button {
  width: 100%;
}

.expense-item {
  flex-direction: column;
  align-items: flex-start;
}

.expense-amount {
  margin-top: 10px;
  align-self: flex-end;
}

.user-info {
  flex-direction: column;
  gap: 10px;
}

.scanner-controls {
  flex-direction: column;
}
}
</style>
</head>
<body>
<div id="authContainer" class="container">
  <form id="loginForm" class="auth-form">
    <h1>Personal Expense Tracker</h1>
    <p>Sign in to your account</p>
    <input type="text" id="loginUsername" placeholder="Username" required />
    <input type="password" id="loginPassword" placeholder="Password" required />
    <button type="submit">Login</button>
    <div class="auth-switch">
      Don't have an account? <a id="showSignup">Sign up</a>
    </div>
  </form>

  <form id="signupForm" class="auth-form" style="display: none;">
    <h1>Create Account</h1>
    <p>Sign up for a new account</p>
    <input type="text" id="signupName" placeholder="Full Name" required />
    <input type="text" id="signupUsername" placeholder="Username" required />
    <input type="email" id="signupEmail" placeholder="Email" required />
    <input type="password" id="signupPassword" placeholder="Password" required />
    <div class="password-strength" id="passwordStrength"></div>
    <input type="password" id="signupConfirmPassword" placeholder="Confirm Password" required />
    <button type="submit">Sign Up</button>
    <div class="auth-switch">
      Already have an account? <a id="showLogin">Sign in</a>
    </div>
  </form>

```

```
</div>
</form>
</div>
```

```
<div id="appContainer" class="container">
  <div class="user-info">
    <div class="user-greeting">Hello, <span id="userName">User</span>!</div>
    <button class="logout-btn" id="logoutBtn">Logout</button>
  </div>
```

```
<h1>Personal Expense Tracker</h1>
```

```
<div id="budgetDisplay">
  <p>Remaining Budget: <span id="remainingBudget">₹0.00</span></p>
</div>
```

```
<form id="budgetForm">
  <input type="number" id="budgetAmount" placeholder="Budget Amount" required />
  <select id="budgetType">
    <option value="weekly">Weekly</option>
    <option value="monthly">Monthly</option>
  </select>
  <button type="submit" class="budget-btn">Set Budget</button>
</form>
```

```
<div id="predictionDisplay">
  <p id="predictionText">Set a budget to see spending predictions</p>
</div>
```

```
<div class="chart-container">
  <h3>Spending by Category</h3>
  <div class="chart-wrapper">
    <canvas id="categoryChart"></canvas>
  </div>
</div>
```

```
<div class="scanner-container">
  <h3>Scan Receipt</h3>
  <p style="text-align: center; color: var(--text-light); margin-bottom: 20px;">
    Use your camera to scan a receipt and automatically extract the amount
  </p>
```

```
<div class="camera-preview" id="cameraPreview">
  <div class="camera-placeholder" id="cameraPlaceholder">
    Camera preview will appear here
  </div>
  <video id="cameraVideo" autoplay playsinline style="display: none;"></video>
  <canvas id="cameraCanvas" style="display: none;"></canvas>
</div>
```

```
<div class="scanner-controls">
  <button id="startCamera" class="scan-btn">Start Camera</button>
  <button id="captureReceipt" class="scan-btn" style="display: none;">Capture & Scan</button>
  <button id="stopCamera" class="delete-btn" style="display: none;">Stop Camera</button>
</div>
```



```

<div class="scanner-loading" id="scannerLoading">
  <div class="spinner"></div>
  <p>Scanning receipt... This may take a few seconds.</p>
</div>

<div class="scanner-result" id="scannerResult" style="display: none;">
  <h4>Scan Results</h4>
  <p id="scannedText"></p>
  <div id="amountSuggestions"></div>
  <button id="useScannedAmount" class="budget-btn" style="margin-top: 10px;">Use This Amount</
button>
</div>
</div>

<form id="expenseForm">
  <input type="number" id="amount" placeholder="Amount" required />
  <input type="text" id="category" placeholder="Category" required />
  <input type="date" id="date" required />
  <button type="submit">Add Expense</button>
</form>

<div id="expenseList">
  <h2>Your Expenses</h2>
  <div id="expensesContainer"></div>
  <button id="deleteAllBtn" class="delete-all-btn">Delete All Expenses</button>
</div>

<h2>Delete an Expense</h2>
<form id="deleteForm">
  <input type="text" id="deleteId" placeholder="Enter Expense ID" required />
  <button type="submit" class="delete-btn">Delete</button>
</form>
</div>

<script>
const API_BASE = 'https://x9vrul3j.execute-api.us-east-1.amazonaws.com/prod';

const USER_STORAGE_KEY = 'expenseTrackerUsers';
let currentBudget = 0;
let budgetType = "weekly";
let expenses = [];
let currentUser = null;
let categoryChart = null;
let cameraStream = null;
let scannedAmount = null;

document.addEventListener('DOMContentLoaded', function() {
  if (!localStorage.getItem(USER_STORAGE_KEY)) {
    localStorage.setItem(USER_STORAGE_KEY, JSON.stringify({}));
  }

  const savedUser = localStorage.getItem('currentUser');
  if (savedUser) {
    currentUser = JSON.parse(savedUser);
    showApp();
  }

```

```

    } else {
      showAuth();
    }
  });

function showAuth() {
  document.getElementById('authContainer').style.display = 'flex';
  document.getElementById('appContainer').style.display = 'none';
}

function showApp() {
  document.getElementById('authContainer').style.display = 'none';
  document.getElementById('appContainer').style.display = 'block';

  document.getElementById('userName').textContent = currentUser.name;

  const userDataKey = `userData_${currentUser.username}`;
  const savedData = localStorage.getItem(userDataKey);

  if (savedData) {
    const userData = JSON.parse(savedData);
    currentBudget = userData.currentBudget || 0;
    budgetType = userData.budgetType || "weekly";
    expenses = userData.expenses || [];

    document.getElementById('budgetType').value = budgetType;
  }

  updateBudgetDisplay();
  renderExpenses();
  updatePrediction();
  updateCategoryChart();
}

function saveUserData() {
  if (currentUser) {
    const userDataKey = `userData_${currentUser.username}`;
    const userData = {
      currentBudget,
      budgetType,
      expenses
    };
    localStorage.setItem(userDataKey, JSON.stringify(userData));
  }
}

function updateBudgetDisplay() {
  const budgetElement = document.getElementById('remainingBudget');
  budgetElement.textContent = `₹${currentBudget.toFixed(2)}`;

  if (currentBudget >= 0) {
    budgetElement.className = 'positive';
  } else {
    budgetElement.className = 'negative';
  }
}

```

```
}
```

```
function updatePrediction() {
  const predictionElement = document.getElementById('predictionText');

  if (expenses.length === 0) {
    predictionElement.textContent = "Add expenses to see spending predictions";
    predictionElement.className = "";
    return;
  }

  if (budgetType === "weekly") {
    const now = new Date();
    const oneWeekAgo = new Date(now);
    oneWeekAgo.setDate(oneWeekAgo.getDate() - 7);

    const recentExpenses = expenses.filter(expense =>
      new Date(expense.date) >= oneWeekAgo
    );

    const weeklySpending = recentExpenses.reduce((sum, expense) => sum + expense.amount, 0);

    if (expenses.length > 0) {
      const firstExpenseDate = new Date(Math.min(...expenses.map(e => new Date(e.date))));
      const weeksOfData = Math.max(1, Math.ceil((now - firstExpenseDate) / (7 * 24 * 60 * 60 * 1000)));
      const totalSpending = expenses.reduce((sum, expense) => sum + expense.amount, 0);
      const averageWeeklySpending = totalSpending / weeksOfData;

      const predictedSpending = weeklySpending > 0 ? weeklySpending : averageWeeklySpending;
      const remainingBudget = currentBudget - predictedSpending;

      if (predictedSpending === 0) {
        predictionElement.textContent = "Not enough spending data for accurate prediction. Add more
expenses to see predictions.";
        predictionElement.className = 'prediction-warning';
      } else if (remainingBudget >= 0) {
        predictionElement.textContent = `Based on your spending patterns (avg: ₹$
{averageWeeklySpending.toFixed(2)}/week), you're on track to stay within budget next week.`;
        predictionElement.className = 'prediction-positive';
      } else {
        predictionElement.textContent = `Warning: Based on your spending patterns (avg: ₹$
{averageWeeklySpending.toFixed(2)}/week), you're projected to exceed your budget by ₹$
{Math.abs(remainingBudget).toFixed(2)} next week.`;
        predictionElement.className = 'prediction-danger';
      }
    }
  }

  } else {
    const now = new Date();
    const oneMonthAgo = new Date(now);
    oneMonthAgo.setMonth(oneMonthAgo.getMonth() - 1);

    const recentExpenses = expenses.filter(expense =>
      new Date(expense.date) >= oneMonthAgo
    );
```

```

const monthlySpending = recentExpenses.reduce((sum, expense) => sum + expense.amount, 0);

if (expenses.length > 0) {
  const firstExpenseDate = new Date(Math.min(...expenses.map(e => new Date(e.date))));
  const monthsOfData = Math.max(1, Math.ceil((now - firstExpenseDate) / (30 * 24 * 60 * 60 * 1000)));
  const totalSpending = expenses.reduce((sum, expense) => sum + expense.amount, 0);
  const averageMonthlySpending = totalSpending / monthsOfData;

  const predictedSpending = monthlySpending > 0 ? monthlySpending : averageMonthlySpending;
  const remainingBudget = currentBudget - predictedSpending;

  if (predictedSpending === 0) {
    predictionElement.textContent = "Not enough spending data for accurate prediction. Add more
expenses to see predictions.";
    predictionElement.className = 'prediction-warning';
  } else if (remainingBudget >= 0) {
    predictionElement.textContent = `Based on your spending patterns (avg: ₹$
{averageMonthlySpending.toFixed(2)}/month), you're on track to stay within budget next month.`;
    predictionElement.className = 'prediction-positive';
  } else {
    predictionElement.textContent = `Warning: Based on your spending patterns (avg: ₹$
{averageMonthlySpending.toFixed(2)}/month), you're projected to exceed your budget by ₹$
{Math.abs(remainingBudget).toFixed(2)} next month.`;
    predictionElement.className = 'prediction-danger';
  }
}
}
}

function updateCategoryChart() {
  const ctx = document.getElementById('categoryChart').getContext('2d');

  if (categoryChart) {
    categoryChart.destroy();
  }

  if (expenses.length === 0) {
    ctx.font = '16px Poppins';
    ctx.fillStyle = '#7f8c8d';
    ctx.textAlign = 'center';
    ctx.fillText('No expenses to display', 300, 150);
    return;
  }

  const categoryTotals = {};
  expenses.forEach(expense => {
    if (categoryTotals[expense.category]) {
      categoryTotals[expense.category] += expense.amount;
    } else {
      categoryTotals[expense.category] = expense.amount;
    }
  });

  const categories = Object.keys(categoryTotals);

```

```
const amounts = Object.values(categoryTotals);
```

```
const backgroundColors = categories.map((_, index) => {  
  const hue = (index * 137.5) % 360;  
  return `hsl(${hue}, 70%, 65%)`;  
});
```

```
categoryChart = new Chart(ctx, {  
  type: 'pie',  
  data: {  
    labels: categories,  
    datasets: [{  
      data: amounts,  
      backgroundColor: backgroundColors,  
      borderColor: '#fff',  
      borderWidth: 2  
    }]  
  },  
  options: {  
    responsive: true,  
    maintainAspectRatio: false,  
    plugins: {  
      legend: {  
        position: 'bottom',  
        labels: {  
          padding: 20,  
          usePointStyle: true,  
          font: {  
            family: 'Poppins',  
            size: 12  
          }  
        }  
      }  
    },  
    tooltip: {  
      callbacks: {  
        label: function(context) {  
          const label = context.label || '';  
          const value = context.raw || 0;  
          const total = context.dataset.data.reduce((a, b) => a + b, 0);  
          const percentage = Math.round((value / total) * 100);  
          return `${label}: ₹${value.toFixed(2)} (${percentage}%)`;  
        }  
      }  
    }  
  }  
});
```

```
function renderExpenses() {  
  const container = document.getElementById('expensesContainer');
```

```
  if (expenses.length === 0) {  
    container.innerHTML = '<div class="empty-list">No expenses added yet. Add your first expense above!  
</div>';  
  }
```

```
    return;  
  }
```

```
const sortedExpenses = [...expenses].sort((a, b) => new Date(b.date) - new Date(a.date));
```

```
container.innerHTML = sortedExpenses.map(expense => `  
  <div class="expense-item">  
    <div class="expense-details">  
      <div class="expense-category">${expense.category}</div>  
      <div class="expense-date">${formatDate(expense.date)}</div>  
      <div class="expense-id">ID: ${expense.id}</div>  
    </div>  
    <div class="expense-amount">-₹${expense.amount.toFixed(2)}</div>  
  </div>  
`).join("");  
}
```

```
function formatDate(dateString) {  
  const options = { year: 'numeric', month: 'short', day: 'numeric' };  
  return new Date(dateString).toLocaleDateString(undefined, options);  
}
```

```
function checkPasswordStrength(password) {  
  const strengthBar = document.getElementById('passwordStrength');
```

```
  strengthBar.className = 'password-strength';
```

```
  if (password.length === 0) {  
    return;  
  }
```

```
  let strength = 0;
```

```
  if (password.length >= 8) strength++;
```

```
  if (/[a-z]/.test(password)) strength++;
```

```
  if (/[A-Z]/.test(password)) strength++;
```

```
  if (/[0-9]/.test(password)) strength++;
```

```
  if (/^[A-Za-z0-9]/.test(password)) strength++;
```

```
  if (strength <= 2) {  
    strengthBar.className = 'password-strength strength-weak';  
  } else if (strength <= 4) {  
    strengthBar.className = 'password-strength strength-medium';  
  } else {  
    strengthBar.className = 'password-strength strength-strong';  
  }  
}
```

```
function getUsers() {  
  return JSON.parse(localStorage.getItem(USER_STORAGE_KEY) || '{}');  
}
```

```
function saveUsers(users) {
  localStorage.setItem(USER_STORAGE_KEY, JSON.stringify(users));
}
```

```
function registerUser(userData) {
  const users = getUsers();

  if (users[userData.username]) {
    return { success: false, message: 'Username already exists' };
  }
```

```
  users[userData.username] = {
    name: userData.name,
    email: userData.email,
    password: userData.password,
    createdAt: new Date().toISOString()
  };
};
```

```
saveUsers(users);
```

```
const userDataKey = `userData_${userData.username}`;
localStorage.setItem(userDataKey, JSON.stringify({
  currentBudget: 0,
  budgetType: "weekly",
  expenses: []
})));
```

```
return { success: true };
}
```

```
function authenticateUser(username, password) {
  const users = getUsers();
  const user = users[username];
```

```
  if (user && user.password === password) {
    return {
      success: true,
      user: {
        username: username,
        name: user.name,
        email: user.email
      }
    };
  }
}
```

```
return { success: false, message: 'Invalid username or password' };
}
```

```
async function startCamera() {
  try {
    cameraStream = await navigator.mediaDevices.getUserMedia({
      video: { facingMode: 'environment' }
    });

    const video = document.getElementById('cameraVideo');
```

```

video.srcObject = cameraStream;
video.style.display = 'block';

document.getElementById('cameraPlaceholder').style.display = 'none';
document.getElementById('startCamera').style.display = 'none';
document.getElementById('captureReceipt').style.display = 'block';
document.getElementById('stopCamera').style.display = 'block';

} catch (error) {
  console.error('Error accessing camera:', error);
  alert('Unable to access camera. Please check permissions.');
```

```

}
```

```

function stopCamera() {
  if (cameraStream) {
    cameraStream.getTracks().forEach(track => track.stop());
    cameraStream = null;
  }
}
```

```

const video = document.getElementById('cameraVideo');
video.srcObject = null;
video.style.display = 'none';
```

```

document.getElementById('cameraPlaceholder').style.display = 'block';
document.getElementById('startCamera').style.display = 'block';
document.getElementById('captureReceipt').style.display = 'none';
document.getElementById('stopCamera').style.display = 'none';
document.getElementById('scannerResult').style.display = 'none';
}
```

```

async function captureAndScan() {
  const video = document.getElementById('cameraVideo');
  const canvas = document.getElementById('cameraCanvas');
  const context = canvas.getContext('2d');
```

```

  canvas.width = video.videoWidth;
  canvas.height = video.videoHeight;
```

```

  context.drawImage(video, 0, 0, canvas.width, canvas.height);
```

```

  document.getElementById('scannerLoading').style.display = 'block';
  document.getElementById('scannerResult').style.display = 'none';
```

```

  try {
    canvas.toBlob(async (blob) => {
      try {
        const { data: { text } } = await Tesseract.recognize(
          blob,
          'eng',
          {
            logger: m => console.log(m)
          }
        );
      }
    });
  }
```

```

  processScannedText(text);
}
```



```

    } catch (error) {
      console.error('OCR Error:', error);
      document.getElementById('scannerLoading').style.display = 'none';
      alert('Error scanning receipt. Please try again.');
```

```

    }
  });
} catch (error) {
  console.error('Capture Error:', error);
  document.getElementById('scannerLoading').style.display = 'none';
  alert('Error capturing image. Please try again.');
```

```

}
}

function processScannedText(text) {
  document.getElementById('scannerLoading').style.display = 'none';

  document.getElementById('scannedText').textContent = `Extracted Text: "${text}"`;

  const amountPattern = /(\${?})(\d{1,3}(?:,\d{3})*(?:\.\d{2})?)/g;
  const amounts = [];
  let match;
```

```

  while ((match = amountPattern.exec(text)) !== null) {
    const amountValue = parseFloat(match[2].replace(/,/g, ""));
    if (!isNaN(amountValue) && amountValue > 0) {
      amounts.push(amountValue);
    }
  }
}
```

```

const uniqueAmounts = [...new Set(amounts)].sort((a, b) => b - a);

const suggestionsDiv = document.getElementById('amountSuggestions');
if (uniqueAmounts.length > 0) {
  suggestionsDiv.innerHTML = `
    <p><strong>Detected Amounts:</strong></p>
    ${uniqueAmounts.map(amount => `
      <button class="amount-suggestion" data-amount="${amount}"
        style="margin: 5px; padding: 8px 12px; background: #e3f2fd; border: 1px solid #bbdefb;
border-radius: 4px; cursor: pointer;">
      ₹${amount.toFixed(2)}
    </button>
  `).join("")}
  `;
};
```

```

document.querySelectorAll('.amount-suggestion').forEach(button => {
  button.addEventListener('click', function() {
    scannedAmount = parseFloat(this.getAttribute('data-amount'));
    document.getElementById('amount').value = scannedAmount.toFixed(2);
    document.getElementById('useScannedAmount').style.display = 'none';
    this.style.background = '#c8e6c9';
    this.style.borderColor = '#a5d6a7';
  });
});
```

```

    scannedAmount = uniqueAmounts[0];
    document.getElementById('useScannedAmount').style.display = 'block';

} else {
    suggestionsDiv.innerHTML = '<p>No amounts detected. Please try again with a clearer image.</p>';
    document.getElementById('useScannedAmount').style.display = 'none';
}

document.getElementById('scannerResult').style.display = 'block';
}

document.getElementById('loginForm').addEventListener('submit', function(e) {
    e.preventDefault();
    const username = document.getElementById('loginUsername').value;
    const password = document.getElementById('loginPassword').value;

    const authResult = authenticateUser(username, password);

    if (authResult.success) {
        currentUser = authResult.user;
        localStorage.setItem('currentUser', JSON.stringify(currentUser));
        showApp();
        this.reset();
    } else {
        alert(authResult.message);
    }
});

document.getElementById('signupForm').addEventListener('submit', function(e) {
    e.preventDefault();
    const name = document.getElementById('signupName').value;
    const username = document.getElementById('signupUsername').value;
    const email = document.getElementById('signupEmail').value;
    const password = document.getElementById('signupPassword').value;
    const confirmPassword = document.getElementById('signupConfirmPassword').value;

    if (password !== confirmPassword) {
        alert('Passwords do not match');
        return;
    }

    if (password.length < 6) {
        alert('Password must be at least 6 characters long');
        return;
    }

    const registrationResult = registerUser({
        name,
        username,
        email,
        password
    });

    if (registrationResult.success) {
        alert('Account created successfully! Please login with your new credentials.');
```

```

    } else {
        alert(registrationResult.message);
    }
});

document.getElementById('signupPassword').addEventListener('input', function() {
    checkPasswordStrength(this.value);
});

document.getElementById('showSignup').addEventListener('click', function() {
    document.getElementById('loginForm').style.display = 'none';
    document.getElementById('signupForm').style.display = 'block';
});

document.getElementById('showLogin').addEventListener('click', showLoginForm);

function showLoginForm() {
    document.getElementById('signupForm').style.display = 'none';
    document.getElementById('loginForm').style.display = 'block';
}

document.getElementById('logoutBtn').addEventListener('click', function() {
    stopCamera();

    currentUser = null;
    localStorage.removeItem('currentUser');
    showAuth();
    showLoginForm();
});

document.getElementById('startCamera').addEventListener('click', startCamera);
document.getElementById('stopCamera').addEventListener('click', stopCamera);
document.getElementById('captureReceipt').addEventListener('click', captureAndScan);

document.getElementById('useScannedAmount').addEventListener('click', function() {
    if (scannedAmount) {
        document.getElementById('amount').value = scannedAmount.toFixed(2);
        this.style.display = 'none';
    }
});

document.getElementById('budgetForm').addEventListener('submit', function(e) {
    e.preventDefault();
    const budgetAmount = parseFloat(document.getElementById('budgetAmount').value);
    budgetType = document.getElementById('budgetType').value;

    if (budgetAmount >= 0) {
        currentBudget = budgetAmount;
        saveUserData();
        updateBudgetDisplay();
        updatePrediction();
        alert('Budget set successfully!');
        this.reset();
    } else {
        alert('Please enter a valid budget amount');
    }
});

```

```
}  
});
```

```
document.getElementById('expenseForm').addEventListener('submit', async function(e) {  
  e.preventDefault();  
  const amount = parseFloat(document.getElementById('amount').value);  
  const category = document.getElementById('category').value;  
  const date = document.getElementById('date').value;
```

```
  currentBudget -= amount;
```

```
  const expenseId = Date.now().toString();  
  const newExpense = {  
    id: expenseId,  
    amount: amount,  
    category: category,  
    date: date  
  };  
  expenses.push(newExpense);  
  saveUserData();  
  
  updateBudgetDisplay();  
  renderExpenses();  
  updatePrediction();  
  updateCategoryChart();  
  
  const res = await fetch(`${API_BASE}/expense`, {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({ amount: Number(amount), category, date })  
  });  
  
  if (res.ok) {  
    alert(' Expense added!');  
    this.reset();  
    scannedAmount = null;  
    document.getElementById('scannerResult').style.display = 'none';  
  } else {  
    currentBudget += amount;  
    expenses.pop();  
    saveUserData();  
    updateBudgetDisplay();  
    renderExpenses();  
    updatePrediction();  
    updateCategoryChart();  
    alert(' Failed to add expense');  
  }  
});
```

```
document.getElementById('deleteForm').addEventListener('submit', async function(e) {  
  e.preventDefault();  
  const expenseId = document.getElementById('deleteId').value;  
  
  const expenseIndex = expenses.findIndex(exp => exp.id === expenseId);
```

```

if (expenseIndex !== -1) {
  const deletedAmount = expenses[expenseIndex].amount;

  currentBudget += deletedAmount;
  expenses.splice(expenseIndex, 1);
  saveUserData();

  updateBudgetDisplay();
  renderExpenses();
  updatePrediction();
  updateCategoryChart();
}

```

```

const res = await fetch(`${API_BASE}/expense`, {
  method: 'DELETE',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ expense_id: expenseId })
});

```

```

if (res.ok) {
  alert('🗑 Expense deleted!');
  this.reset();
} else {
  if (expenseIndex !== -1) {
    currentBudget -= deletedAmount;
    expenses.push({
      id: expenseId,
      amount: deletedAmount,
      category: expenses[expenseIndex].category,
      date: expenses[expenseIndex].date
    });
    saveUserData();
    updateBudgetDisplay();
    renderExpenses();
    updatePrediction();
    updateCategoryChart();
  }
  alert('Failed to delete');
}
});

```

```

document.getElementById('deleteAllBtn').addEventListener('click', async function() {
  if (expenses.length === 0) {
    alert('No expenses to delete!');
    return;
  }

```

```

  if (!confirm('Are you sure you want to delete ALL expenses? This action cannot be undone.')) {
    return;
  }

```

```

const totalAmount = expenses.reduce((sum, expense) => sum + expense.amount, 0);

```

```

currentBudget += totalAmount;
expenses = [];

```

```

saveUserData();

updateBudgetDisplay();
renderExpenses();
updatePrediction();
updateCategoryChart();

try {
  const res = await fetch(`${API_BASE}/expenses`, {
    method: 'DELETE',
    headers: { 'Content-Type': 'application/json' }
  });

  if (res.ok) {
    alert('🗑️ All expenses deleted successfully!');
  } else {
    currentBudget -= totalAmount;
    alert('Failed to delete all expenses from server. Local changes rolled back.');
```

updateBudgetDisplay();

renderExpenses();

updatePrediction();

updateCategoryChart();

}

```

  } catch (error) {
    currentBudget -= totalAmount;
    alert('Network error. Failed to delete all expenses from server. Local changes rolled back.');
```

updateBudgetDisplay();

renderExpenses();

updatePrediction();

updateCategoryChart();

}

```

  });
</script>
</body>
</html>

```